A Service-Oriented Task & Deadline Management Platform

Dr. Neelima Gogineni,
Assistant Professor, Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad-500043, Telangana, India. Mail: dr.neelima@klh.edu.in.

MADHUPRAVEENA
Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad-500043, Telangana, India.
madhupraveena09@gmail.com

HEMA THULASI
Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad-500043, Telangana, India.
hemathulasi.ht567@gmail.com

RANAN MAGDALA
Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad-500043, Telangana, India.
2420030744@cse.klh.edu.in

TEAM 11 | SOA IEEE CONFERENCE

Abstract—Task and deadline management is an important requirement for students, professionals, and collaborative teams that handle multiple activities with different priorities and due dates. Manual tracking across notes, messages, and disconnected applications can make it difficult to maintain task visibility and respond to approaching deadlines. This paper presents a Service-Oriented Task & Deadline Management Platform that centralizes task organization and automates deadline-related notifications. The platform is developed using React.js, Node.js, Express.js, MongoDB, JWT, Node-Cron, Nodemailer, Socket.IO, and Tailwind CSS. A REST-based backend exposes task and authentication operations, while scheduled processing checks configured reminder conditions. Email notifications and real-time events provide additional channels for notifying users. Docker, GitHub, and Postman support deployment consistency, source-code collaboration, and API validation. The paper describes the proposed architecture, service responsibilities, data flow, implementation approach, and evaluation strategy. Quantitative performance results are not asserted in this draft; they must be populated from repeatable tests of the completed application.

Index Terms—Service-Oriented Architecture, Task Management, Deadline Management, RESTful Services, JWT Authentication, Scheduled Jobs, Real-Time Notifications, MongoDB.

I. INTRODUCTION

Task management is a common requirement in academic and professional environments. Individuals frequently handle assignments, meetings, project milestones, examinations, and personal activities that must be completed within specific time limits. As the number of tasks increases, relying on memory or scattered records can lead to missed deadlines, duplicated effort, and limited visibility into work progress.

A digital task-management system provides a centralized location to record activities, assign priorities, monitor completion status, and review upcoming deadlines. However, a basic task list may not be sufficient when users need timely reminders and immediate updates. A practical solution should combine task operations with authentication, scheduled background processing, and notification delivery.

The proposed Service-Oriented Task & Deadline Management Platform addresses these needs through a modular web application. The frontend is developed using React.js and Tailwind CSS. The backend uses Node.js and Express.js to expose REST APIs, while MongoDB stores application data. JWT is used for authenticated access. Node-Cron supports periodic deadline checks, Nodemailer handles email delivery, and Socket.IO enables real-time notification events.

The system is organized around separate responsibilities so that user interaction, business logic, persistence, scheduling, and communication can be developed and tested independently. The main objectives are to provide centralized task management, protect user-specific resources, monitor configured deadlines, and notify users through email and real-time application updates.

The task lifecycle begins when a user records an activity and assigns a due date. The task can then be reviewed, prioritized, updated, and marked as completed. A clear distinction between priority and status helps users identify important pending work while retaining an accurate view of progress. The platform's central data store provides a consistent view of these attributes across sessions.

Task data should remain understandable over time. Creation and update timestamps provide a basic record of when an activity entered the system and when it was last modified. Due dates should be stored in a consistent format and converted for display according to the user's timezone. These practices reduce ambiguity when tasks are viewed from different devices or when the application is deployed on servers configured with a different system timezone.

Task management also involves maintaining consistency between the information displayed in the browser and the records stored by the server. After a successful create or update operation, the frontend should use the response from the backend to refresh the affected task data. If an operation fails, the interface should preserve the user's input where practical and display a meaningful error message. This feedback loop helps prevent users from assuming that an unsuccessful operation has been saved.

The project follows a layered approach in which presentation logic is kept separate from persistence and notification logic. This separation supports maintainability because changes to the visual interface do not necessarily require changes to the database model. Similarly, the notification mechanism can be modified without redesigning the task-entry interface, provided that the service contracts remain stable.

The intended workflow is applicable to academic assignments, project milestones, personal reminders, and team deliverables. The current project scope focuses on task organization and deadline notification. Advanced collaboration features, such as assigning tasks to multiple users or maintaining shared project boards, are considered extensions rather than assumed capabilities of the present implementation.

The expected users include individuals who manage academic schedules, project work, and personal responsibilities. The platform is not intended to replace a full enterprise project-management suite in its current scope. Instead, it establishes a compact and extensible foundation that can be enhanced with collaboration, calendar synchronization, and analytics features in later versions.

The interaction model is intentionally simple: users authenticate, access their own task collection, and perform task operations through the interface. The backend remains the authority for data access and business rules. This separation is important because hiding a button in the frontend is not a substitute for enforcing authorization on the server.

An important design consideration is the distinction between a task record and a notification event. The task record represents the current state of work, while a notification represents an attempt to communicate a time-sensitive update. Keeping these concepts separate makes it possible to update a task without losing the history of reminder activity. If notification history is not part of the implemented schema, it can be maintained as a future enhancement.

The scope of the platform is focused on personal task and deadline coordination. A task is treated as a manageable unit of work with descriptive information and a target completion date. The platform provides a consistent lifecycle for recording a task, changing its status as work progresses, and retaining the updated state for later access. This creates a single source of task information for the authenticated user.

Task priority and task status serve different purposes in this workflow. Priority communicates the relative importance of an activity, whereas status indicates its current stage of completion. Keeping these attributes separate allows a user to identify important pending work without confusing urgency with progress. Due-date information further supports sorting and deadline-based filtering.

From a service-oriented perspective, the system separates the interface from the backend operations that enforce application rules. The frontend is responsible for presenting data and collecting user actions, but it is not trusted to decide whether a user may access a particular task. The backend validates requests, checks identity, and performs database operations. Notification services are invoked through backend-controlled workflows.

The platform also separates user-facing operations from time-triggered processing. A user may create or update a task at any time, but deadline evaluation occurs according to the scheduler configuration. This reduces the need for the browser to remain active and makes reminder behavior consistent across user sessions. The scheduled process must use a consistent timezone policy so that a deadline is not interpreted differently by the browser, API server, and scheduler.

From an operational perspective, the platform should provide clear feedback when a task is saved, updated, or removed. Validation messages should identify missing or incorrectly formatted values. Task lists can be ordered by due date or priority to help users identify work that requires attention. These interface behaviors are usability considerations and should be checked against the final running implementation.

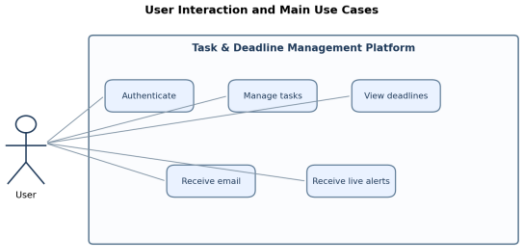


Fig. 1. User interaction and primary use cases of the proposed platform.

II. LITERATURE SURVEY

Service-oriented architecture (SOA) structures software as a collection of services that expose defined interfaces and perform specific responsibilities. A service-oriented approach can reduce tight dependencies between application components and allow individual modules to evolve without redesigning the entire system. In web applications, these principles are commonly reflected in resource-oriented APIs and modular backend components.

Representational State Transfer (REST) is an architectural style for distributed systems. RESTful services use standard HTTP methods and resource identifiers to support operations such as creating, retrieving, updating, and deleting application data. This model is appropriate for task-management applications because users and tasks can be represented as resources and accessed through consistent API contracts.

Token-based authentication is widely used in client–server applications. JSON Web Token (JWT) defines a compact representation of claims that can be digitally signed. A backend can verify a token before allowing access to protected operations. Authentication alone does not guarantee authorization; the application must also verify that a task belongs to the authenticated user.

Background scheduling is useful when an application must perform checks independently of user interaction. A scheduled process can inspect stored task deadlines and initiate reminder actions when configured conditions are met. Email provides an asynchronous notification channel, while Socket.IO supports event-driven communication between a server and connected clients.

Existing task applications commonly provide task creation, status tracking, and calendar-based reminders. The proposed platform combines these capabilities with a service-oriented backend that integrates REST APIs, JWT middleware, a scheduled deadline-monitoring component, email delivery, and real-time notification events. The contribution of this project is the integration of these functions in a single modular application rather than the introduction of a new authentication or scheduling algorithm.

A. Comparison of Related Approaches

Area	Common approach	Proposed platform focus
Task organization	Manual lists or basic CRUD applications	Centralized task records with status and due dates
Authentication	Session or token-based access	JWT-protected API operations
Deadline reminders	Manual checking or fixed reminders	Scheduled deadline evaluation using Node-Cron
Notifications	Email or in-app alerts in isolation	Nodemailer and Socket.IO integration
Application design	Monolithic or tightly coupled modules	Separated frontend, API, persistence and notification roles

III. PROBLEM DEFINITION AND SYSTEM REQUIREMENTS

The problem addressed by this project is the difficulty of maintaining a reliable and unified view of tasks and deadlines when information is distributed across multiple tools. Users may overlook upcoming due dates, fail to update task status, or miss important changes when reminders depend entirely on manual action.

A. Functional Requirements

- User registration and login with protected access to personal task data.
- Create, view, update, and delete task records.
- Maintain task details such as title, description, due date, priority, and status, subject to the implemented schema.
- Run scheduled checks for tasks that meet configured reminder conditions.
- Send deadline-related email messages and publish real-time notification events.

B. Non-Functional Requirements

- Maintainability through modular service responsibilities and clear API contracts.
- Security through password protection, JWT verification, input validation, and resource ownership checks.
- Usability through a responsive interface and clear task-status presentation.
- Reliability through controlled reminder execution, error handling, and notification logging where implemented.

C. System Actors and Use

The primary actor is an authenticated user who manages personal tasks through the web interface. The scheduler acts as an internal system component rather than a human actor. It periodically evaluates stored task information and triggers notification workflows. The backend mediates all data access so that client-side controls cannot bypass server-side authorization.

IV. PROPOSED SYSTEM AND ARCHITECTURE

D. Service-Oriented Design Rationale

The platform is described as service-oriented because its major responsibilities are separated behind clear interfaces. The frontend does not directly access the database; it communicates with the backend through HTTP endpoints. Authentication is applied as middleware, task operations are handled by application logic, and notification work is delegated to scheduling and communication components. This separation reduces direct dependencies between modules and allows each responsibility to be tested independently.

Service boundaries in this project are logical boundaries within one application rather than a claim that every component is deployed as an independent microservice. For a student project, a modular Node.js application can provide the benefits of separation without introducing the operational complexity of a distributed microservice environment. The architecture can later be decomposed into separately deployed services if scale, team ownership, or availability requirements justify that change.

E. Design Principles

The proposed design follows separation of concerns, least privilege, and explicit interface contracts. Separation of concerns keeps presentation, data access, scheduling, and communication logic distinct. Least privilege requires each authenticated user to access only the resources permitted by the application. Explicit interface contracts define the request fields, response format, and error behavior expected by the frontend and other backend modules.

The architecture also distinguishes immediate user operations from background work. Creating or editing a task should return a response without waiting for an email provider to complete delivery. Deadline reminders are processed asynchronously by the scheduled workflow. This prevents a temporary email delivery problem from blocking ordinary task management.

The proposed platform follows a client–server architecture in which the browser-based frontend communicates with backend services through REST APIs. The backend coordinates authentication, task processing, database operations, and notification services. Scheduled deadline checks operate as a background responsibility and do not require the user to keep the task page open.

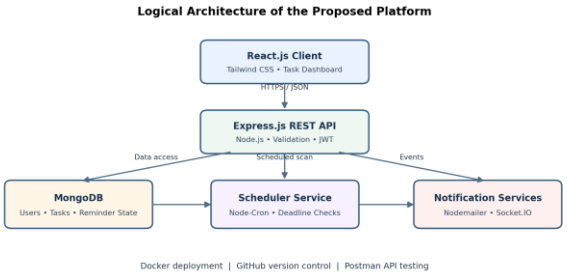


Fig. 2. Logical architecture of the Service-Oriented Task & Deadline Management Platform.

A. Presentation Layer

The presentation layer is developed using React.js. It provides the interface through which users authenticate, create tasks, review task lists, modify task details, and view notifications. Tailwind CSS is used to create responsive layouts and consistent component styling. The frontend communicates with backend endpoints using HTTP requests and processes JSON responses.

B. API and Application Layer

Node.js provides the server-side runtime and Express.js defines the REST API routes and middleware pipeline. The API layer receives client requests, validates input, verifies authentication where required, invokes the relevant application logic, and returns structured responses. Separating route handling from data and notification responsibilities improves readability and testability.

C. Authentication Service

JWT-based middleware protects routes that require an authenticated user. After successful login, the server issues a signed token. The client supplies the token with subsequent protected requests, commonly through the Authorization header using the Bearer scheme. The backend verifies token validity and must additionally enforce ownership checks before allowing access to a task.

D. Data Persistence Layer

MongoDB stores user and task information. A typical task record may contain an identifier, owner reference, title, description, due date, priority, status, and timestamps. The final schema must match the actual application implementation. Database queries for user-facing operations should be scoped to the authenticated user's identity.

E. Scheduler and Notification Layer

Node-Cron is used to schedule periodic deadline checks. The scheduler queries relevant task records and evaluates whether a task falls within the configured reminder window. Nodemailer is responsible for sending email notifications through a configured transport. Socket.IO emits events to connected clients so that notification changes can appear without a full page refresh.

The scheduler should avoid repeatedly sending the same reminder on every execution. A sent-reminder marker or notification log can record delivery state. Timezone handling, retry rules, and behavior for overdue tasks should be explicitly configured and verified during testing.

An additional consideration is concurrency. If two scheduler executions overlap, both may identify the same task as eligible for notification. A robust implementation can reduce this risk by recording a reminder state before or immediately after dispatch, using atomic database updates where appropriate, and ensuring that repeated processing does not create duplicate messages. The chosen approach should be documented according to the actual implementation.

V. METHODOLOGY AND IMPLEMENTATION

F. Task Lifecycle and State Management

A task typically moves through a lifecycle that begins when it is created, remains active while work is pending or in progress, and reaches a terminal completed state when the user finishes it. The exact status values must match the implementation. A task may also become overdue when its due date passes while it remains incomplete. Overdue status can be derived from the due date and current status rather than stored as a separate state, provided that the application consistently applies the same rule.

Priority and status should be treated as separate attributes. Priority represents relative importance, while status represents progress. Due date represents the target completion time. Keeping these dimensions separate supports sorting and filtering, such as displaying high-priority pending tasks or tasks due within the next day. The interface should make these attributes visible without presenting derived information as if it were a user-entered value.

G. Data Validation and API Error Handling

Validation should be performed on the server even when the frontend already validates form fields. Client-side validation improves usability, but requests can be sent without using the interface. The backend should verify that required fields are present, that values have the expected types, and that date values can be parsed. It should also reject malformed identifiers and unsupported status or priority values.

Consistent error handling improves the reliability of frontend integration. Authentication failures should be distinguished from authorization failures, missing records, validation errors, and unexpected server errors. The API should return appropriate HTTP status codes and a predictable JSON response structure. Error messages should be useful for the client while avoiding disclosure of internal stack traces, database details, or secrets.

Real-time communication also requires connection management. A client may disconnect because of a network interruption or browser closure. The application should not treat a disconnected socket as a failure of the underlying task operation. When the client reconnects, it can retrieve the latest task and notification state through the REST API. This combination of REST retrieval and event-driven updates provides a practical model for maintaining a current interface.

The modular structure supports incremental testing. The API can be tested independently through Postman, database behavior can be checked using representative user and task records, and the scheduler can be tested with controlled due dates. Email and Socket.IO components can be validated separately before an end-to-end test confirms that a task deadline produces the expected notification workflow.

The data model can be represented through a user collection and a task collection. Each task stores an owner reference so that user-specific operations can be scoped to the authenticated account. Additional notification metadata may be stored when the implementation requires a record of reminder attempts or delivery status. The diagram represents a logical relationship; field names should be aligned with the final MongoDB schema.

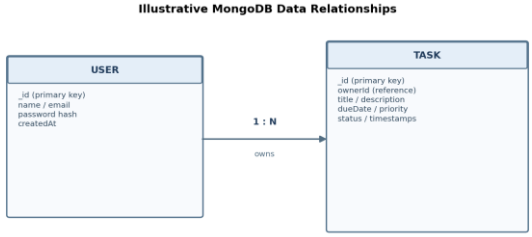


Fig. 3. Illustrative user–task data relationship for MongoDB persistence.

The user–task relationship supports data isolation by design. A query for a task list should include the authenticated owner identifier, and a request to update a task should verify ownership before applying changes. The task identifier alone should never be treated as proof that the requester is authorized to modify a record.

Database indexes may be considered for fields frequently used in queries, such as owner identifiers and due dates. Index selection should be based on observed query patterns and the size of the stored collection. For an academic prototype, correctness and clear ownership checks are the primary requirements; performance tuning can be evaluated as the volume of task records grows.

H. Reminder Policy and Idempotency

The reminder policy defines which tasks are eligible for notification. It may consider the task due date, current time, completion status, user preferences, and whether a reminder has already been delivered. The reminder interval should be configurable so that the team can change the behavior without rewriting the task-management interface. The implementation should clearly define whether it sends one reminder, multiple reminders, or an overdue notification.

Idempotency is important because scheduled jobs execute repeatedly. If a task remains inside a reminder window for several scheduler runs, the system should not send an identical message on every run unless repeated reminders are intentional. A sent-at timestamp, reminder status, or separate notification record can be used to identify completed delivery attempts. The check and update should be designed to reduce race conditions if multiple scheduler instances can run concurrently.

I. Real-Time Event Handling

Socket.IO supports event-based updates between the server and connected clients. A client can subscribe to the notification events relevant to its authenticated account. The server should associate socket connections with verified user identity and emit private notifications only to the appropriate user or room. Broadcasting task details to all connected clients would expose user-specific information and should be avoided.

Real-time events should complement, not replace, the REST API. A socket connection may be interrupted when a browser closes or a network changes. On reconnect, the client can retrieve the current task and notification state through the API. This combination allows the application to recover from missed events instead of relying on every notification being delivered to a continuously connected browser.

The implementation is organized into frontend development, backend API development, database integration, authentication, and notification processing. Each component communicates through defined interfaces. This modular

organization supports independent testing and reduces the need for unrelated modules to share internal implementation details.

A. Task Management Workflow

When a user submits a task operation, the React client sends a request to the corresponding REST endpoint. Express.js middleware validates the request and checks the JWT for protected operations. The application layer applies task-specific rules and interacts with MongoDB. The server then returns a response that the frontend uses to refresh the relevant task view.

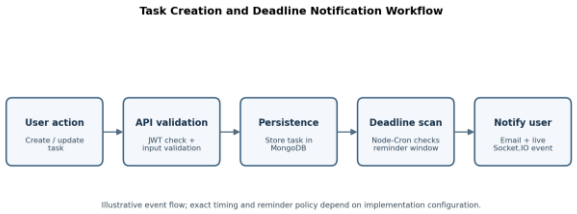


Fig. 4. Illustrative task creation and deadline notification workflow.

B. API Design

The REST API is organized around authentication and task resources. The following endpoints illustrate a possible interface and should be updated to match the routes implemented in the project.

API responses should use a consistent JSON structure so that the frontend can handle successful and unsuccessful requests predictably. For example, a successful task creation response may return the created task and a status code indicating that a resource was created. A request with invalid input should return a client-error status and a message describing the validation failure. The precise response schema should be documented from the actual server implementation.

Method	Example endpoint	Operation
POST	/api/auth/register	Register user
POST	/api/auth/login	Authenticate user
GET	/api/tasks	List authenticated user's tasks
POST	/api/tasks	Create a task
PUT	/api/tasks/{id}	Update a task
DELETE	/api/tasks/{id}	Delete a task

C. Technology Stack

Component	Technology	Responsibility
Frontend	React.js, Tailwind CSS	UI and responsive presentation
Backend	Node.js, Express.js	REST API and business logic
Database	MongoDB	User and task persistence
Authentication	JWT	Token-based route protection
Scheduling	Node-Cron	Periodic deadline checks
Email	Nodemailer	Email reminder delivery
Real-time	Socket.IO	Live notification events
DevOps/testing	Docker, GitHub, Postman	Containers, versioning, API tests

D. Development and Testing Tools

GitHub is used for source-code version control and team collaboration. Postman supports endpoint testing by allowing developers to send requests, inspect status codes, validate response bodies, and test authentication headers. Docker can package the application and runtime dependencies into repeatable environments. Configuration values such as database URLs, JWT secrets, and email credentials should be stored in environment variables and excluded from public repositories.

Database operations are designed around the authenticated user's identity. When the API retrieves a task list, the query should filter records by the owner identifier extracted from the verified token. Update and delete operations should combine the task identifier with the owner identifier rather than trusting a client-supplied user value. This approach reduces the risk of insecure direct object references.

Input validation is applied at the API boundary before a request is processed. Required fields should be checked for presence and valid type, and date values should be normalized consistently before they are compared with scheduler time. Error responses should communicate whether a request failed because of invalid input, missing authentication, insufficient authorization, a missing task, or an unexpected server-side condition.

The deadline-monitoring process can be represented as a recurring sequence: load eligible tasks, compare each due date with the configured reminder window, determine whether a reminder has already been issued, and dispatch the required notification. A reminder record or timestamp can be used to make this operation idempotent. The exact frequency of execution and reminder interval should be selected according to project requirements.

A deployment arrangement may run the frontend, API server, scheduler process, and database as separate logical components. Docker can be used to package application processes and define service dependencies. In a production environment, the database may be hosted separately from the application containers. The actual deployment topology should be documented from the team's Docker configuration rather than inferred from this conceptual view.

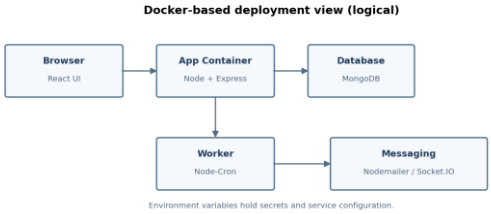


Fig. 5. Logical deployment view of the platform and its supporting services.

The deployment model should define how configuration is shared across environments. Development, testing, and demonstration environments may use different database endpoints and email credentials, but the application should read these values through the same configuration interface. This reduces the risk of accidentally using production credentials during local development and makes the deployment procedure easier to reproduce.

Before a demonstration, the team should verify the complete startup sequence. The frontend should load successfully, the backend should establish a database connection, protected API calls should accept a valid token, and the scheduler should initialize without duplicate background jobs. The notification workflow should be tested with a controlled task deadline and a test recipient so that the team can confirm both the generated email and the in-app event.

Containerization does not remove the need for operational monitoring. A container may start successfully while an external dependency, such as the database or mail server, remains unavailable. Health checks and clear startup errors can help identify these conditions. For a larger deployment, the scheduler may be separated from the API process to prevent multiple application instances from independently running the same scheduled job.

The platform's service boundaries also create opportunities for future scaling. The API can be scaled to handle more user requests, while notification processing can be managed separately according to delivery volume. If the application later supports shared workspaces or team assignments, authorization rules can be extended to include project membership and role permissions without changing the basic task resource model.

A containerized setup can simplify local development by ensuring that team members use compatible runtime versions and service configurations. The application should document the required environment variables, startup commands, exposed ports, and service dependencies. Sensitive values must be supplied through a local environment file or deployment secret store and should not be committed to GitHub.

Deployment validation should include a clean startup, successful database connection, API health check, and verification that the scheduler and notification components initialize correctly. The team should also test restart behavior so that a process restart does not unintentionally cause previously delivered reminders to be sent again.

Operational logs can record application startup, API failures, scheduler runs, and notification outcomes. Logs are useful for debugging the system during the project demonstration. They should avoid recording passwords, authentication tokens, or private email credentials. A consistent log format also makes it easier to trace a task from creation to reminder delivery.

Email delivery depends on the availability and configuration of the selected mail transport. The application should handle delivery errors without interrupting unrelated task operations. Socket.IO events should be emitted only to the intended user or authenticated connection scope. These design considerations are important because notifications may contain personal task titles or deadline information.

The notification sequence begins with a task being created or updated through the API. At a later scheduled execution, the deadline-monitoring component evaluates eligible tasks and triggers a notification action when the configured condition is satisfied. The email and real-time channels are independent delivery mechanisms; a failure in one channel should be logged and handled without corrupting the underlying task record.

To evaluate the notification subsystem, the team can record the scheduled execution time, the task due date, the time at which the reminder condition is detected, and the time at which the delivery attempt completes. The difference between these timestamps provides an estimate of scheduler delay and notification processing time. Email delivery latency can vary because it depends on the configured mail provider and network conditions.

Testing should also consider boundary conditions. A task whose due date is just outside the reminder window should not be selected, while a task that enters the configured window should be selected on a subsequent scheduler run. Completed or deleted tasks should not continue to generate reminders unless the project explicitly defines such behavior. These cases help verify that reminder rules are applied consistently.

The dashboard is intended to summarize the user's work rather than replace the detailed task view. Counts such as total tasks, pending tasks, completed tasks, and tasks due soon can help users understand their workload at a glance. The values shown in the final application should be derived from database records and should update after task operations. Any dashboard image included in the final paper should be captured from the running project.

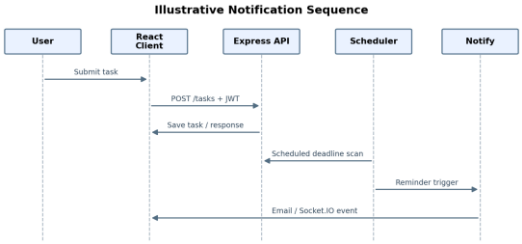


Fig. 7. Illustrative sequence of task processing and deadline notification.

During validation, the team should test both normal and exceptional paths. Examples include submitting an empty task title, entering an invalid date, using an expired token, attempting to update a task owned by another account, and simulating an unavailable email service. Testing these cases provides stronger evidence of correctness than demonstrating only the successful creation of a task.

VI. RESULTS AND DISCUSSION

D. Test Strategy and Evaluation Metrics

A systematic test strategy should cover unit-level validation of individual functions, integration testing between the API and database, and end-to-end testing of the complete task and reminder workflow. API tests can be executed through Postman, while scheduler tests can use controlled task due dates and a shortened test interval. Email tests should use a designated test recipient rather than a real user's address.

Useful evaluation metrics include API response time, successful task-operation rate, scheduler detection delay, email delivery success rate, and real-time event delivery delay. These metrics should be collected over repeated test runs and reported with the test environment, sample size, and measurement method. A single successful demonstration is evidence of a working path, but it is not sufficient to claim a general performance improvement or production-level reliability.

E. Boundary and Negative Test Scenarios

Boundary tests should include tasks due exactly at the reminder threshold, tasks just outside the threshold, tasks with past due dates, and tasks that are completed before the scheduled scan. Negative tests should include invalid or expired JWTs, malformed task identifiers, missing required fields, invalid date strings, and attempts to access another user's task. These scenarios help establish that the system behaves correctly beyond the simplest successful use case.

Notification failure tests should include an unavailable mail transport, a disconnected Socket.IO client, and a scheduler restart. The expected behavior should be defined for each case. For example, an email failure should be recorded for later investigation and should not delete or corrupt the task. A disconnected browser should be able to recover current state after reconnecting through the REST API.

F. Threats to Validity and Current Limitations

The evaluation of a prototype may be affected by the size of the test dataset, the network used during testing, the mail provider, and the hardware on which the application runs. Results obtained on a small local dataset should not be generalized to a large production workload. Email delivery time is also influenced by external provider behavior and cannot be attributed solely to the application code.

The current scope focuses on individual task and deadline management. Unless implemented separately, the platform should not be described as supporting shared workspaces, team assignment, calendar synchronization, mobile push notifications, or enterprise role management. These capabilities are possible future extensions, but they require additional data models, authorization rules, and testing.

The platform is evaluated through functional testing of authentication, task operations, scheduled deadline checks, email delivery, and real-time notification behavior. No execution logs, screenshots from the running application, or measured performance values were supplied while preparing this paper. Accordingly, this section defines the evaluation criteria and does not claim numerical results that have not been measured.

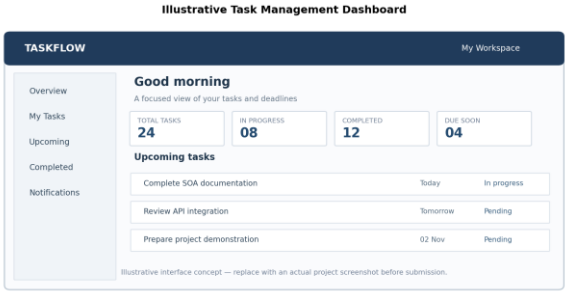


Fig. 6. Illustrative dashboard concept; replace with an actual application screenshot for final submission.

A. Functional Evaluation

Test case	Expected observation
Registration/login	Valid credentials authenticate; invalid credentials are rejected.
Protected routes	Missing or invalid tokens cannot access protected resources.
Task operations	Authorized users can create, retrieve, update, and delete their own tasks.
Ownership control	A user cannot access or modify another user's task.
Deadline scheduler	Tasks matching the configured reminder window are detected.
Email notification	The configured email transport sends the reminder successfully.
Real-time event	Connected clients receive the relevant Socket.IO event.

B. Discussion

The architecture combines synchronous request–response processing with asynchronous deadline and notification workflows. REST APIs handle direct user actions, while the scheduler performs periodic checks independently of browser activity. Email notifications provide an external communication channel, and Socket.IO supports updates to connected clients. The effectiveness of these mechanisms depends on correct scheduler configuration, data consistency, network availability, email transport settings, and error-handling logic.

C. Security and Reliability

Passwords should be stored using a suitable password-hashing function rather than plaintext. JWT signing secrets and email credentials must not be committed to source control. Protected endpoints must validate both token authenticity and resource ownership. Production deployment should use HTTPS, restrict cross-origin requests to trusted origins, validate incoming data, and avoid logging sensitive credentials or tokens. Notification processing should record delivery state and handle transient failures without unintentionally generating duplicate messages.

VII. CONCLUSION AND FUTURE SCOPE

This paper presented a Service-Oriented Task & Deadline Management Platform that integrates task organization, authenticated REST APIs, database persistence, scheduled deadline monitoring, email reminders, and real-time notifications. The system uses React.js, Node.js, Express.js, MongoDB, JWT, Node-Cron, Nodemailer, Socket.IO, and Tailwind CSS, with Docker, GitHub, and Postman supporting deployment and development activities. Its modular structure separates the user interface, API layer, persistence, scheduling, and notification responsibilities.

Future enhancements may include shared team workspaces, task assignment, recurring tasks, calendar synchronization, configurable reminder preferences, role-based access control, analytics dashboards, and a progressive web application. Further evaluation should measure API response time, scheduler delay, email delivery success, notification latency, and behavior under concurrent users. Such results should be reported only after repeatable testing on the completed application.

REFERENCES

[1] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[2] R. Fielding and J. Reschke, "Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content," RFC 7231, IETF, Jun. 2014.

[3] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.

[4] React Team, "React Documentation." [Online]. Available: https://react.dev/learn

[5] OpenJS Foundation, "Node.js Documentation." [Online]. Available: https://nodejs.org/docs/latest/api/

[6] Express.js, “Express Documentation.” [Online]. Available: https://expressjs.com/

[7] MongoDB, Inc., “MongoDB Manual.” [Online]. Available: https://www.mongodb.com/docs/manual/

[8] Socket.IO, “Socket.IO Documentation.” [Online]. Available: https://socket.io/docs/v4/

[9] Nodemailer, “Nodemailer Documentation.” [Online]. Available: https://nodemailer.com/

[10] Docker, Inc., “Docker Documentation.” [Online]. Available: https://docs.docker.com/

[11] OWASP Foundation, “OWASP API Security Top 10—2023.” [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x11-t10/

[12] M. Nottingham, “HTTP Semantics,” RFC 9110, Internet Engineering Task Force, Jun. 2022.

[13] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, Internet Engineering Task Force, Aug. 2018.

[14] OWASP Foundation, “Password Storage Cheat Sheet.” [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[15] Postman, “Postman Learning Center.” [Online]. Available: https://learning.postman.com/